

Lab1 实验报告

一、思考题

Thinking 1.1

使用 gcc，传参-M elf32-x86-64 为指定 elf 文件格式，-S 表示生成详细输出，main.o 指定解析的文件

```
git@23373428:~/23373428 (lab1)$ gcc -E main.c > main.i
git@23373428:~/23373428 (lab1)$ gcc -c main.i
git@23373428:~/23373428 (lab1)$ gcc main.o -o main
git@23373428:~/23373428 (lab1)$ objdump -M elf32-x86-64 -S main.o

main.o:          文件格式 elf64-x86-64

Disassembly of section .text:

0000000000000000 <main>:
 0:  f3 0f 1e fa          endbr64
 4:  55                   push    %rbp
 5:  48 89 e5             mov     %rsp,%rbp
 8:  48 8d 05 00 00 00 00 lea     0x0(%rip),%rax      # f <main+0xf>
 f:  48 89 c7             mov     %rax,%rdi
12:  e8 00 00 00 00       call    17 <main+0x17>
17:  b8 00 00 00 00       mov     $0x0,%eax
1c:  5d                   pop     %rbp
1d:  c3                   ret
```

```
00000000000001149 <main>:
1149:  f3 0f 1e fa          endbr64
114d:  55                   push    %rbp
114e:  48 89 e5             mov     %rsp,%rbp
1151:  48 8d 05 ac 0e 00 00 lea     0xeac(%rip),%rax    # 2004 <_IO_stdin_used+0x4>
1158:  48 89 c7             mov     %rax,%rdi
115b:  e8 f0 fe ff ff       call    1050 <puts@plt>
1160:  b8 00 00 00 00       mov     $0x0,%eax
1165:  5d                   pop     %rbp
1166:  c3                   ret
```

call 对应的地址由 e800000000 变为 e8ffeffff，证明完成了链接

使用 mips-linux-gnu-，传参-M elf32-x86-64 为指定 elf 文件格式，-S 表示生成详细输出，main.o 和 main 指定解析的文件

```

git@23373428:~/23373428 (lab1)$ mips-linux-gnu-objdump -M elf32-mips -S main.o

main.o:      文件格式 elf32-tradbigmips

Disassembly of section .text:

00000000 <main>:
   0:  27bdffe0      addiu    sp,sp,-32
   4:  afbf001c      sw       ra,28(sp)
   8:  afbe0018      sw       s8,24(sp)
  c:  03a0f025      move    s8,sp
 10:  3c1c0000      lui     gp,0x0
 14:  279c0000      addiu    gp,gp,0

```

```

git@23373428:~/23373428 (lab1)$ mips-linux-gnu-objdump -M elf32-mips -S main

main:      文件格式 elf32-tradbigmips

Disassembly of section .init:

004004e4 <_init>:
 4004e4:  3c1c0002      lui     gp,0x2
 4004e8:  279c7b2c      addiu    gp,gp,31532
 4004ec:  0399e021      addu     gp,gp,t9
 4004f0:  27bdffe0      addiu    sp,sp,-32
 4004f4:  afbc0010      sw       gp,16(sp)

```

main 函数入口地址链接为 004004e4

Thinking 1.2

使用自己编写的 readelf

```

git@23373428:~/23373428 (lab1)$ ./tools/readelf/readelf target/mos
0:0x0
1:0x80020000
2:0x80021330
3:0x80021348
4:0x80021360
5:0x800216d0
6:0x0
7:0x0
8:0x0
9:0x0
10:0x0
11:0x0

```

使用系统的 readelf 工具:

```
git@23373428:~/23373428 (lab1)$ readelf -S target/mos
There are 12 section headers, starting at offset 0x2010:

节头:
[Nr] Name                Type              Addr             Off             Size            ES Flg Lk Inf
Al
[ 0]                      NULL              00000000 000000 000000 00      0  0
0
[ 1] .text                  PROGBITS          80020000 0000c0 001330 00 WAX 0  0
16
[ 2] .reginfo               MIPS_REGINFO      80021330 0013f0 000018 18   A  0  0
4
[ 3] .MIPS.abiflags         MIPS_ABIFLAGS     80021348 001408 000018 18   A  0  0
8
[ 4] .rodata.str1.4         PROGBITS          80021360 001420 000364 01 AMS 0  0
4
[ 5] .rodata                PROGBITS          800216d0 001790 0001e0 00   A  0  0
16
[ 6] .pdr                   PROGBITS          00000000 001970 000240 00      0  0
4
[ 7] .comment               PROGBITS          00000000 001bb0 000026 01  MS 0  0
1
[ 8] .gnu.attributes        GNU_ATTRIBUTES    00000000 001bd6 000010 00      0  0
1
[ 9] .symtab                SYMTAB            00000000 001be8 0002a0 10      10 21
4
[10] .strtab                STRTAB            00000000 001e88 00011a 00      0  0
1
[11] .shstrtab              STRTAB            00000000 001fa2 00006e 00      0  0
1
```

可以正常解析 ELF 内核文件

Makefile 内容:

```
1 %.o: %.c
2     $(CC) -c $<
3
4 .PHONY: clean
5
6 readelf: main.o readelf.o
7     $(CC) $^ -o $@
8
9 hello: hello.c
10    $(CC) $^ -o $@ -m32 -static -g
11
12 clean:
13    rm -f *.o readelf hello
```

使用 readelf -h:

```
git@23373428:~/23373428 (lab1)$ readelf -h ./tools/readelf/readelf
ELF 头:
  Magic:      7f 45 4c 46 02 01 01 00 00 00 00 00 00 00 00
  类别:                      ELF64
  数据:                      2 补码, 小端序 (little endian)
  Version:                  1 (current)
  OS/ABI:                    UNIX - System V
  ABI 版本:                  0
  类型:                      DYN (Position-Independent Executable file)
)
```

可见 Makefile 中编译 hello 的时候使用了 -m32, 编译 readelf 的时候没有, 生成了 ELF64 格式的 readelf, 但是编写时使用了 32 位参数结果, 导致不能解析自身

Thinking 1.3

第一阶段：**bootloader** 加载到内存中，属于硬件层面，发生在上电地址；

第二阶段：**bootloader** 加载操作系统内核，发生在不是上电地址的另一个内存特定位置，再移交控制权，由内核启动代码启动并找到入口地址，系统开始运转；

在实验中，模拟器代为完成了第一步，因此保证启动入口地址符合内存布局即可。

二、实验难点

本次作业中的难点主要在于理论知识较为复杂，在此之前没有接触过类似体系的知识，没掌握具体可行的学习方法，只得逐字逐句研读琢磨，还不能构建知识网络。因此对于实验也只是摸着石头过河，懵懵懂懂，根据代码要求进行编写，而不能先有框架、再有功能。

对此，尽管实验代码部分做完了，我仍然大量阅读了相关知识与实验的博客，汲取吸收前人的学习经验与思考模式，由写代码时的知识层面跃迁到体系层面，再次思考本次实验。